

Race Condition Demonstration (make race vs make fixed)

Introduction

A race condition occurs when multiple threads access and modify shared data simultaneously without proper synchronization. This may produce inconsistent and unpredictable results because thread execution order cannot be guaranteed.

The objective of this experiment was to demonstrate how synchronization mechanisms such as mutex locks can prevent race conditions.

Race Version (make race)

The race version executes multiple threads that increment a shared counter without synchronization.

Compile and run:

```
make race  
race.exe
```

Example Output:

```
Running race condition demo...  
  
Expected Counter = 400000  
  
Actual Counter = 287143  
  
Race condition detected.
```

Observation

The expected counter value should be **400000**, however the actual result is lower because multiple threads attempt to update the shared variable simultaneously.

Since there is no synchronization:

- Threads overwrite each other's updates

- Execution order becomes unpredictable
- Results vary each time the program runs

Fixed Version (make fixed)

The fixed version introduces mutex synchronization.

Compile and run:

```
make fixed
fixed.exe
```

Example Output:

```
Running synchronized version...

Expected Counter = 400000

Actual Counter = 400000

Synchronization successful.
```

Synchronization Method Used

Mutex protection was implemented using:

```
pthread_mutex_lock(&lock);

counter++;

pthread_mutex_unlock(&lock);
```

The mutex ensures that only one thread accesses the shared counter at a time.

Comparison

Feature	Race Version	Fixed Version
Synchronization	No	Yes

Feature	Race Version	Fixed Version
Counter Accuracy	Incorrect	Correct
Thread Safety	No	Yes
Output Stability	Unpredictable	Consistent

Conclusion

The experiment demonstrates that race conditions occur when multiple threads modify shared resources without synchronization. By introducing mutex locks, shared data becomes protected and thread execution becomes safe and predictable.

This experiment highlights the importance of synchronization mechanisms in concurrent operating system environments.